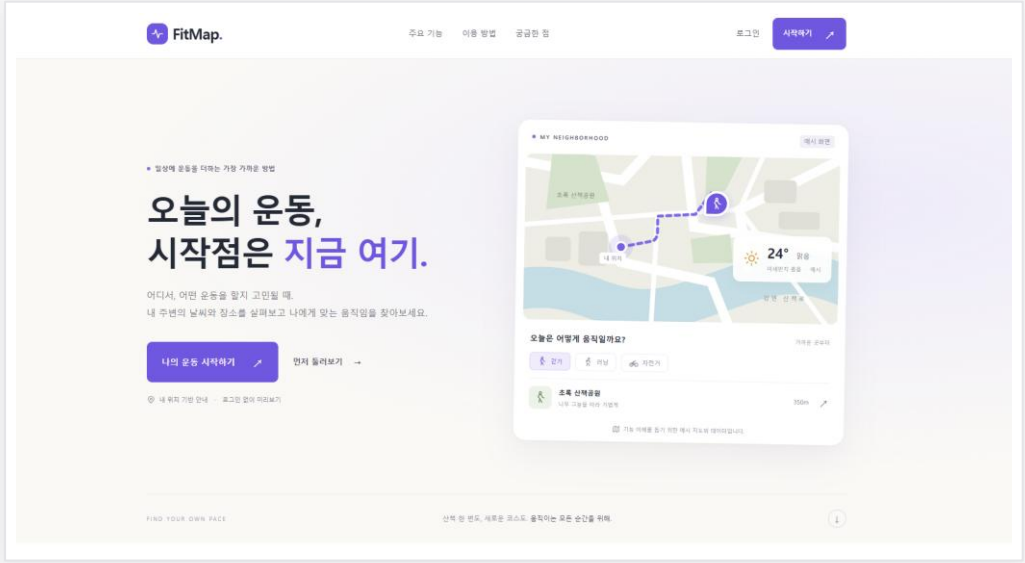
2026 캡스톤디자인 기획서

날씨 API, 지도 API 기반
위치정보 운동 웹사이트

실시간 날씨와 대기질, 카카오 지도를 결합해 지금 내 주변의 운동 장소를 추천합니다.



사용 기술

기상청 단기예보 API | 에어코리아 API | Kakao Maps API | Kakao Local API

React | Node.js | Spring Boot | H2 Database | Docker

팀명	지도교수	팀장	팀원
FitMap	정환익	주승재	신동환, 이정윤, 최우혁

목차

01	프로젝트 개요	팀 구성, 개발 목적, 유사 제품 비교와 차별점
02	시스템 구성	처리 구성, 시스템 구성도, 외부 API 연동, 날씨 데이터 처리
03	시스템 운영 설계	운영 구성도, 추천 규칙, 백엔드, 위치와 장소, 이동 경로, 즐겨찾기, UI
04	시스템 활용 방안	날씨별 사용 시나리오
05	기대 효과 및 결론	기대 효과와 결론

프로젝트 개요

프로젝트명

날씨 API와 지도 API를 활용한 위치정보 운동 웹사이트

개발 내용

날씨 API와 지도 API로 사용자 위치의 우천, 자외선, 미세먼지 같은 날씨 정보를 확인하고, 운동하기 적절한 곳을 지도에 표시합니다.

프로젝트 목표	운동 효율을 높이고, 날씨의 영향을 받지 않는 운동 장소를 추천합니다.
사용 대상	운동을 하고자 하는 모든 사람
기대 효과	• 날씨, 미세먼지, 주변 운동장 정보를 한 곳에서 확인 • 실시간 날씨와 미세먼지로 운동하기 적합한 장소 추천 • 현재 위치를 활용해 주변 운동 장소를 빠르게 탐색

팀 구성 및 담당 분야

이름	역할	담당 업무
주승재	팀장	자료 정리, 보고서 작성, UI 설계, 메인 페이지 제작
신동환	팀원	UI 설계, 백엔드 구축, API 연동
이정윤	팀원	UI 설계, 로그인 페이지, 운동정보 페이지
최우혁	팀원	UI 설계, 회원가입 페이지, 주변장소 페이지

지도교수 **정환익**　　팀명 **FitMap**

팀원별 프로젝트 진행 내용

주승재

팀장

담당 업무

- 자료 정리
- 보고서 작성
- UI 설계
- 메인 페이지 제작

관련 업무 소개

프로젝트 목표에 맞춰 UI를 설계하고 메인 페이지를 제작합니다. 필요한 데이터 수집과 자료 정리, 보고서 작성을 맡습니다.

업무 추진 방법

팀원들과 UI 설계 기준을 맞추고, 수집한 자료를 정리해 기획서와 보고서에 반영합니다.

업무 추진 내용

서비스 첫 화면(내 위치 기반 소개, 주요 기능, 이용 안내)을 제작하고 기획서를 작성했습니다.

프로젝트 사용 내용

React | UI 설계 | 보고서

팀원별 프로젝트 진행 내용

신동환

팀원

담당 업무

- UI 설계
- 백엔드 구축
- API 연동

관련 업무 소개

UI 설계와 함께 서버를 구축하고, 외부 API를 연동해 서비스에 필요한 데이터를 제공합니다.

업무 추진 방법

화면과 주고받을 데이터 구조를 먼저 정하고, 외부 API 응답을 서버에서 가공해 전달합니다.

업무 추진 내용

Spring Boot와 Node.js로 백엔드를 구성했습니다. Node.js는 카카오, 기상청, 에어코리아 API를 연동하고, Spring Boot는 회원과 즐겨찾기, 이동 경로를 H2에 저장합니다.

프로젝트 사용 내용

Spring Boot | Node.js | H2 Database | Docker

팀원별 프로젝트 진행 내용

이정윤

팀원

담당 업무

- UI 설계
- 로그인 페이지
- 운동정보 페이지

관련 업무 소개

UI를 설계하고 로그인 페이지와 운동정보 페이지를 제작합니다.

업무 추진 방법

공통 UI 기준에 맞춰 화면을 설계한 뒤, 화면 단위로 구현하고 백엔드 API와 연결합니다.

업무 추진 내용

로그인 화면(이메일, 비밀번호, 로그인 상태 유지)과 날씨와 미세먼지 기반 운동 정보 화면을 구현합니다.

프로젝트 사용 내용

React | UI 설계 | API 연동

팀원별 프로젝트 진행 내용

최우혁

팀원

담당 업무

- UI 설계
- 회원가입 페이지
- 주변장소 페이지

관련 업무 소개

UI를 설계하고 회원가입 페이지와 주변장소 페이지를 제작합니다.

업무 추진 방법

카카오 지도 API의 장소 검색 결과를 지도와 목록으로 보여주는 구조로 설계합니다.

업무 추진 내용

회원가입 화면(이름, 닉네임, 이메일, 비밀번호 확인, 약관 동의)과 현재 위치 주변 운동 장소 화면을 구현합니다.

프로젝트 사용 내용

React | Kakao Maps API | Kakao Local API | Geolocation API

시스템 개발 목적과 필요성

문제 1

정보가 흩어져 있다

야외 운동에는 기온, 우천, 미세먼지, 자외선 지수 등 많은 정보가 필요하지만 한 번에 보기 어렵습니다.

문제 2

장소를 직접 찾아야 한다

운동할 곳에 비가 오면, 직접 검색해서 다른 운동 장소를 찾아야 하는 번거로움이 있습니다.

해결

위치 기반으로 날씨와 운동 장소를 함께 보여주고, 날씨에 맞는 곳을 추천합니다

날씨와 자외선	기상청 API
미세먼지	에어코리아 API
주변 운동 장소	Kakao Maps, Local API

유사 제품 비교: 런 어웨이 vs FitMap

유사 제품

런 어웨이

날씨 기반 러닝 타이밍 추천 서비스

주요 기능

- 기온, 강수, 습도, 바람, 미세먼지 5가지 요소 분석
- 하루 중 가장 뛰기 좋은 최적의 시간 알림
- 러닝에 맞춘 날씨 분석

핵심 질문

“언제 뛰면 좋을까?”

비교 항목	런 어웨이	FitMap
핵심 질문	언제 뛰면 좋을까?	**지금 어디서 운동할까?**
추천 결과	하루 중 최적의 러닝 시간대	**현재 위치 주변의 운동 장소 (야외, 실내)**
날씨 데이터	기온, 강수, 습도, 바람, 미세먼지 5요소 분석	**기상청(기온, 강수, 자외선)과 에어코리아 (PM10, PM2.5)**
악천후 대응	날씨가 좋아지는 시간대를 기다림	**실내 운동 장소로 즉시 전환해 추천**
지도와 장소	시간 추천 중심 (장소 탐색은 주 기능 아님)	**카카오 지도에 주변 장소 마커와 거리순 목록**
운동 종목	러닝 중심	**운동 종목 선택 후 종목별 장소 검색**

FitMap 차별화 포인트

시간 추천에서 장소 추천으로

런 어웨이 하루 중 뛰기 좋은 "시간"을 알려줌

FitMap **지금 내 위치 주변의 운동 "장소"를 바로 추천**

Geolocation API | Kakao Local API

악천후 시 실내 자동 전환

런 어웨이 날씨가 나쁘면 더 나은 시간대를 기다려야 함

FitMap **비, 폭염, 미세먼지 나쁨이면 실내 장소로 전환해 추천**

기상청 API | 에어코리아 API | 프론트엔드 추천 로직

날씨와 지도를 한 화면에

런 어웨이 날씨 분석 결과 위주로 확인

FitMap **날씨와 대기 카드, 지도 마커, 장소 목록을 한 화면에**

Kakao Maps API | React

다양한 운동 종목

런 어웨이 러닝에 맞춘 분석

FitMap **운동 종목을 고르면 종목별 장소를 검색**

Kakao Local 키워드 검색 | Node.js

시스템 처리 구성



데이터 생성

사용자 디바이스

- 웹 브라우저
- 활동 데이터 생성
- GPS 위치 좌표 수집

데이터 연결

네트워크와 외부 API

- 지도, 공공데이터
- 외부 API 연동

데이터 처리

Node.js, Spring Boot

- 외부 데이터 가공 (Node.js)
- 회원, 즐겨찾기, 이동 경로 저장
- 비밀번호 해시, 쿠키 세션

서비스 제공

프론트엔드 (React)

- 지도 기반 정보 시각화
- 날씨 기반 운동 추천
- 사용자 웹, 앱 UI

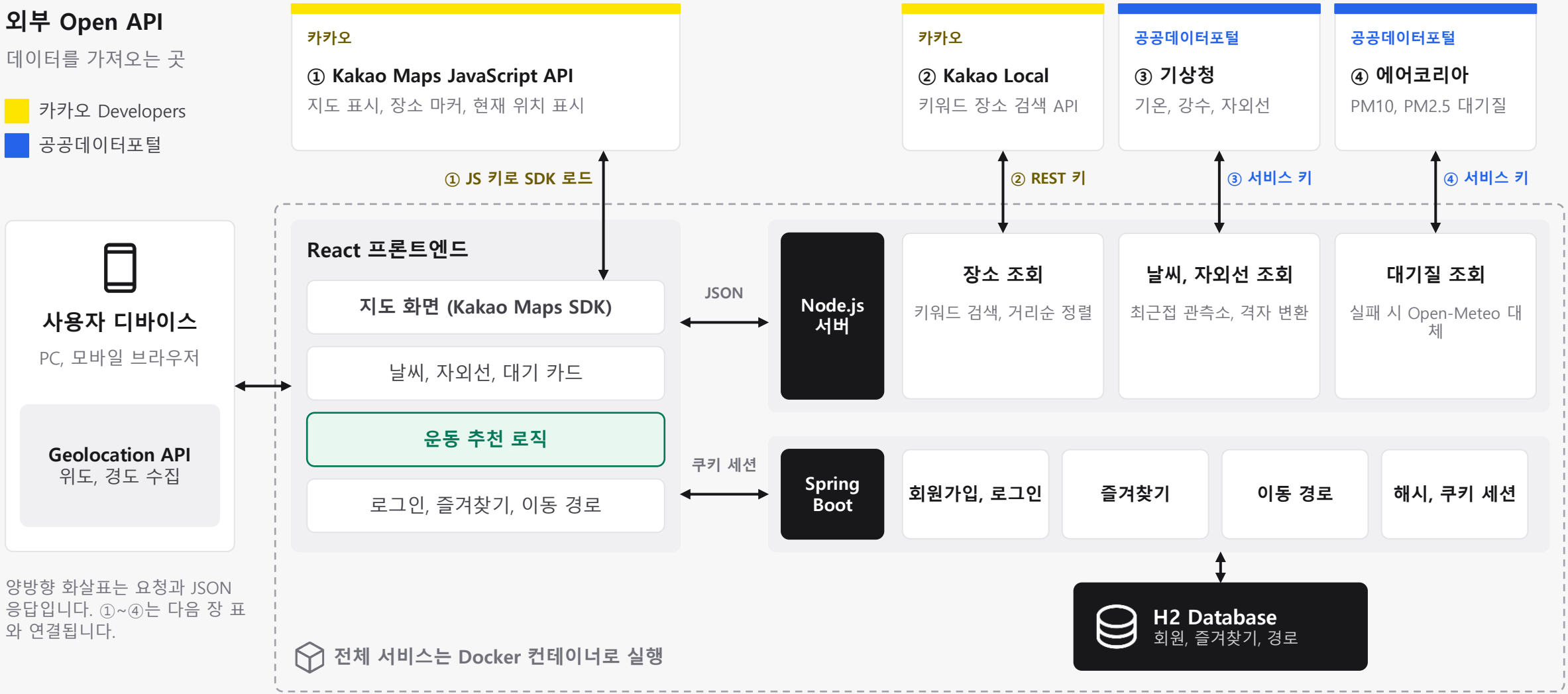
사용자 디바이스에서 생성된 데이터가 API를 거쳐 서버에서 처리되고, 웹 화면으로 제공됩니다.

시스템 구성도: API 연결 구조

외부 Open API

데이터를 가져오는 곳

- 카카오 Developers
- 공공데이터포털



양방향 화살표는 요청과 JSON 응답입니다. ①~④는 다음 장 표와 연결됩니다.

외부 API 연동 상세: 어디서 무엇을 가져오나

API	제공처	호출 위치	요청 값	받아오는 데이터	사용 화면
⓪ Geolocation API `navigator.geolocation`	웹 브라우저 표준	**프론트엔드**	위치 권한 허용	**위도(lat), 경도(lng)**	모든 추천의 기준 좌표
① Kakao Maps JavaScript API `dapi.kakao.com/v2/maps/sdk.js`	카카오 Developers	**프론트엔드**	JavaScript 키, 지도 중심 좌표	**지도 타일, 마커, 인포윈도우**	지도 화면
② Kakao Local API `dapi.kakao.com/v2/local/search/keyword.json`	카카오 Developers	**Node.js 서버**	query(운동장, 헬스장, 공원 등), x, y 좌표, radius	**장소명, 주소, 좌표, 거리, 전화번호**	주변 장소 목록, 지도 마커
③ 기상청 관측, 예보, 자외선 `apis.data.go.kr (기상청)`	공공데이터포털 (기상청)	**Node.js 서버**	최근접 AWS 관측소, nx, ny(격자 좌표)	**실측 기온, 강수, 바람, 자외선지수**	날씨, 자외선 카드, 운동 추천
④ 에어코리아 대기오염정보 `apis.data.go.kr/B552584/ArpltnInforInqireSvc`	공공데이터포털 (한국환경공단)	**Node.js 서버**	stationName(측정소명)	**PM10, PM2.5 농도와 등급 (실패 시 Open-Meteo 추정치)**	대기 정보 카드, 운동 추천

카카오 REST 키와 공공데이터 서비스 키는 Node.js 서버에만 보관하고, 브라우저에는 JavaScript 키만 노출합니다.

날씨 데이터 처리: 최근접 관측소와 격자 변환

최근접 AWS 관측소 선택

```
for (const row of rows(text)) {
  if (!/^\d{12}$/.test(row[0]) || row.length < 15 ||
      row[0] < earliest || row[0] > current) continue
  const temperature = Number(row[8])
  if (!Number.isFinite(temperature) || temperature < -90 || temperature > 60) continue
  if (!latest.has(row[1]) || latest.get(row[1])![0] < row[0]) latest.set(row[1], row)
}
const nearest = stations
  .map(station => ({ station, distance: distanceKm(lat, lon, station) }))
  .filter(item => item.distance <= 50 && latest.has(item.station.id))
  .sort((a, b) => a.distance - b.distance)[0]
```

기상청 격자 변환

```
export function toGrid(latitude: number, longitude: number) {
  const rad = Math.PI / 180
  const re = 6371.00877 / 5
  const slat1 = 30 * rad
  const slat2 = 60 * rad
  const sn = Math.log(Math.cos(slat1) / Math.cos(slat2)) /
    Math.log(Math.tan(Math.PI / 4 + slat2 / 2) / Math.tan(Math.PI / 4 + slat1 / 2))
  const sf = Math.pow(Math.tan(Math.PI / 4 + slat1 / 2), sn) * Math.cos(slat1) / sn
  const ro = re * sf / Math.pow(Math.tan(Math.PI / 4 + 38 * rad / 2), sn)
  const ra = re * sf / Math.pow(Math.tan(Math.PI / 4 + latitude * rad / 2), sn)
  const theta = (longitude - 126) * rad * sn
  return { nx: Math.floor(ra * Math.sin(theta) + 43.5),
           ny: Math.floor(ro - ra * Math.cos(theta) + 136.5) }
}
```

최근접 관측소 선택

내 위치와 가장 가까운 기상청 관측소의 최신 실측값을 고릅니다.

1. 최근 20분 자료만 사용
2. 기온 -90~60℃ 밖 이상치 제거
3. 반경 50km 이내 최근접 관측소 선택

격자 변환

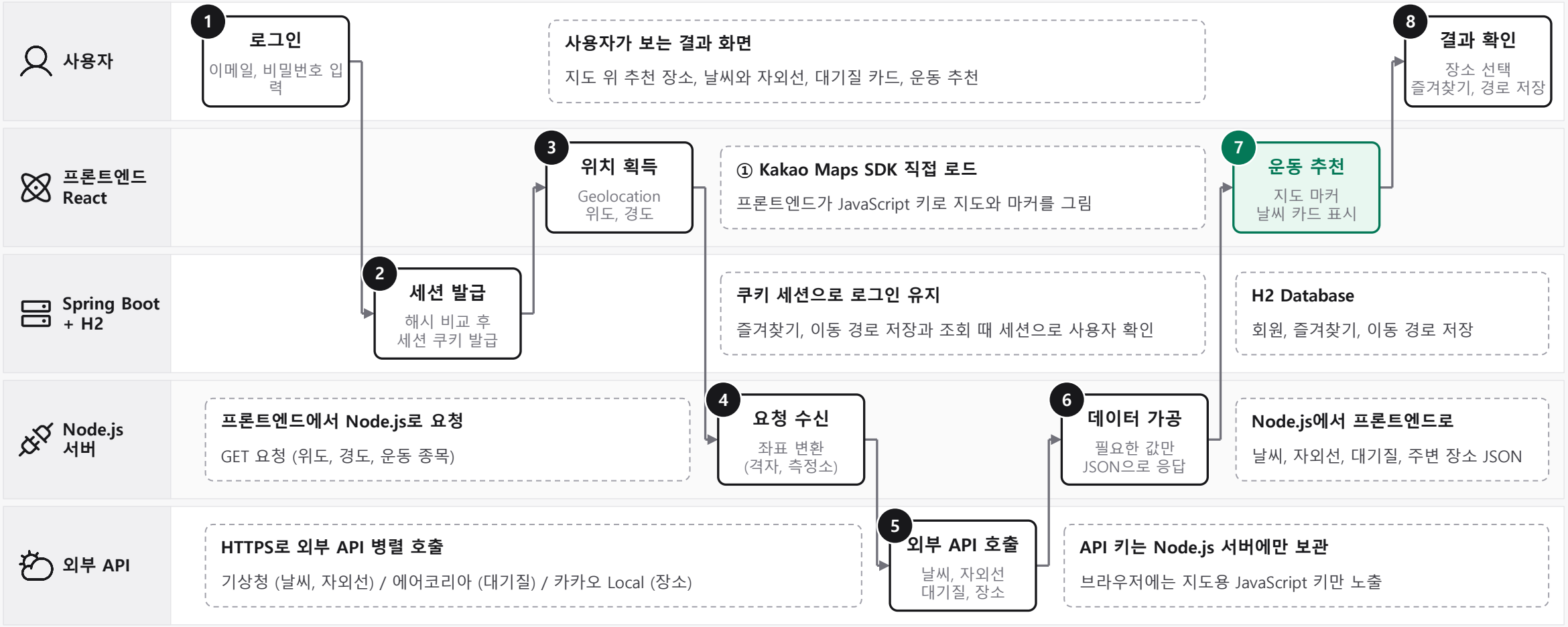
위경도를 기상청 예보 격자(nx, ny)로 변환합니다.

1. Lambert 정각원추 투영 공식
2. 격자 간격 5km
3. 단기예보 API 호출에 꼭 필요한 입력값

시스템 구성 설명

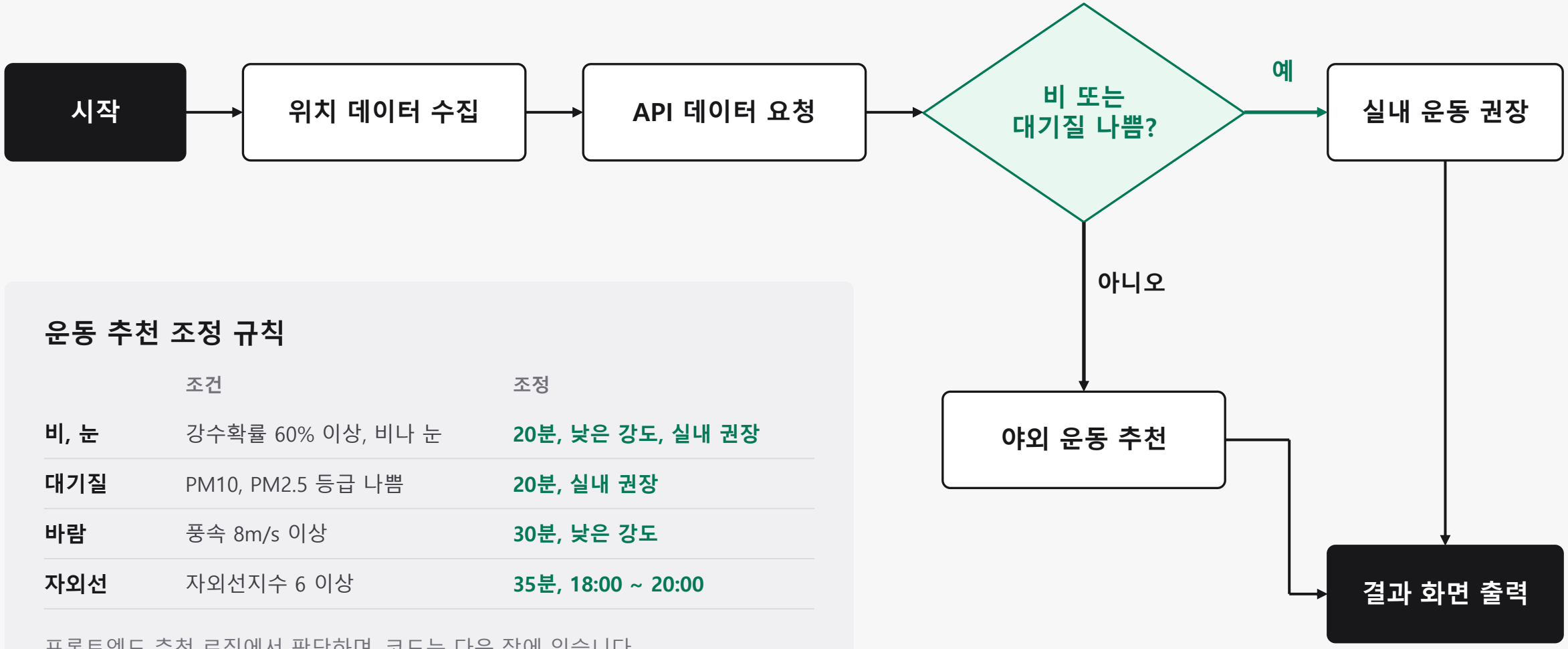
레이어	목적	구성 요소
데이터 수집 레이어	사용자의 현재 위치를 확보	브라우저 Geolocation API (위도, 경도)
외부 데이터 연동 레이어	날씨, 자외선, 대기질, 장소 데이터를 가져옴	Node.js 서버에서 기상청, 에어코리아, Kakao Local API 호출
서버와 저장 레이어	외부 데이터 가공, 회원 데이터 저장과 인증	Node.js, Spring Boot, H2 Database, 비밀번호 해시, 쿠키 세션
사용자 인터페이스 레이어	정보를 화면에 표시하고 운동을 추천	React, Kakao Maps JavaScript API, 운동 추천 로직

시스템 운영 구성도: 요청 처리 흐름



외부 데이터는 Node.js, 회원 데이터는 Spring Boot가 맡고, 운동 추천(7단계)은 프론트엔드에서 판단합니다. 판단 기준은 다음 장에 있습니다.

시스템 작동 플로우 차트



운동 추천 조정 규칙

	조건	조정
비, 눈	강수확률 60% 이상, 비나 눈	**20분, 낮은 강도, 실내 권장**
대기질	PM10, PM2.5 등급 나쁨	**20분, 실내 권장**
바람	풍속 8m/s 이상	**30분, 낮은 강도**
자외선	자외선지수 6 이상	**35분, 18:00 ~ 20:00**

프론트엔드 추천 로직에서 판단하며, 코드는 다음 장에 있습니다.

운동 추천 규칙

운동 추천 규칙

```
const badAir = [air?.pm10.grade, air?.pm25.grade].some(grade => grade?.includes('나쁨'))
const rainy = (weather.precipitation ?? 0) >= 60 ||
  /비|눈|소나기|강수/.test(weather.condition)
const preferIndoor = outdoor && (rainy || badAir)
if (outdoor && rainy) {
  minutes = Math.min(minutes, 20); intensity = '낮음'; time = '실내 운동 권장'
}
if (outdoor && (weather.wind ?? 0) >= 8) { minutes = Math.min(minutes, 30); intensity = '낮음' }
if (outdoor && (uv?.value ?? 0) >= 6) {
  minutes = Math.min(minutes, 35)
  if (!preferIndoor) time = '18:00 ~ 20:00'
}
if (outdoor && badAir) { minutes = Math.min(minutes, 20); time = '실내 운동 권장' }
return { minutes, intensity, calories: Math.round(minutes * base.kcalPerMinute), time, message }
```

앞 장의 조정 규칙을 프론트엔드에서 그대로 구현한 코드입니다.

날씨에 맞춘 운동 조정

날씨, 대기질, 자외선으로 운동 시간, 강도, 시간대를 조정합니다.

1. 비, 미세먼지는 20분, 실내 권장
2. 바람 8m/s 이상은 30분
3. UV 6 이상은 35분

실내 권장일 때는 자외선 규칙이 추천 시간대를 덮어쓰지 않습니다. 예상 칼로리는 운동 시간과 종목별 분당 소모량으로 계산합니다.

백엔드 구성: Spring Boot와 Node.js

Node.js

외부 데이터 서버

카카오, 기상청, 에어코리아 API에서 주변 장소와 날씨, 자외선, 대기질을 조회해 프론트엔드에 전달합니다.

주변 장소	카카오 Local API
날씨, 자외선	기상청 API
대기질	에어코리아 API

REST API · JSON 응답 · API 키 보관

Spring Boot

회원 데이터 서버

회원가입, 로그인, 즐겨찾기, 이동 경로를 관리하고 데이터를 H2 데이터베이스에 저장합니다.

회원가입, 로그인	비밀번호 해시 저장
즐겨찾기	장소 저장과 조회
이동 경로	경로 저장과 조회

H2 Database · 쿠키 세션 · 비밀번호 해시

React 프론트엔드가 두 서버에 각각 요청하고, 받은 정보를 화면에 표시하고 운동 추천에 활용합니다.

외부 API 호출 래퍼와 대기질 폴백

외부 API 호출 래퍼

```
async function upstream<T>(url: URL, headers?): Promise<T> {
  let response = await fetch(url, { headers, signal: AbortSignal.timeout(12000) })
  if (response.status >= 500 && response.status < 600) {
    await new Promise(resolve => setTimeout(resolve, 300))
    response = await fetch(url, { headers, signal: AbortSignal.timeout(12000) })
  }
  if (!response.ok) {
    const message = response.status === 401 || response.status === 403
      ? '외부 API 인증이 거절되었습니다. …'
      : response.status === 429 ? '외부 API 요청 한도를 초과했습니다. …'
      : '외부 API 서버가 일시적으로 응답하지 않습니다. …'
    throw new ApiError(502, message)
  }
  return await response.json() as T
}
```

외부 API 호출 래퍼

모든 외부 API 호출이 거치는 Node.js 서버의 공통 함수입니다.

1. 12초 타임아웃
2. 5xx 오류면 0.3초 뒤 1회 재시도
3. 401, 403, 429별 한국어 오류 안내

대기질 폴백

```
async function resilientAirQuality(config: Config, lat: number, lon: number) {
  try { return await airQuality(config, lat, lon) }
  catch (originalError) {
    if (originalError instanceof ApiError &&
        originalError.status === 503) throw originalError
    try { return await modelAirQuality(lat, lon) }
    catch { throw originalError }
  }
}
```

대기질 폴백

에어코리아 조회가 실패하면 Open-Meteo 추정치로 대체합니다.

1. 키 미설정(503)은 폴백 없이 오류
2. 추정치에는 경고 문구를 함께 표시
3. 5분 캐시와 함께 동작

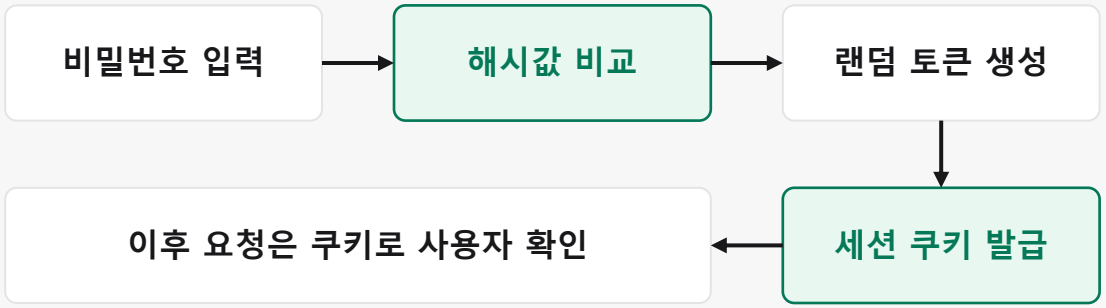
인증과 실행 환경

회원가입과 로그인

회원가입

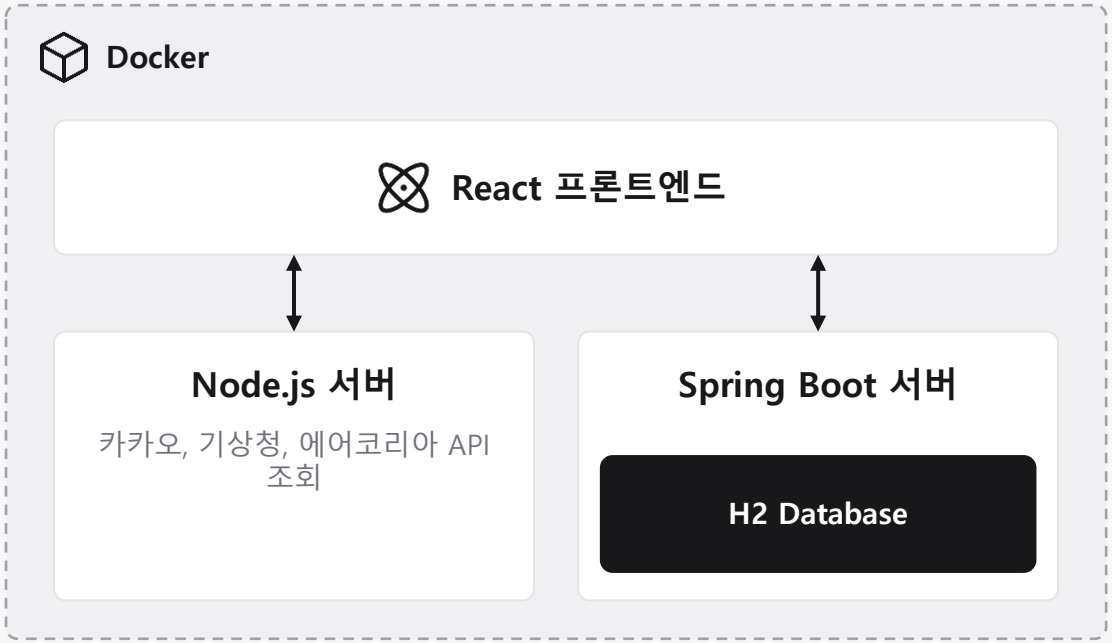


로그인



비밀번호와 세션 토큰은 해시값으로만 저장되고, 로그인 상태는 쿠키 기반 세션으로 유지됩니다.

Docker 실행 환경



프론트엔드, Node.js, Spring Boot를 Docker 컨테이너로 실행해 개발 환경과 상관없이 같은 방식으로 서비스를 띄웁니다.

로그인 세션과 안전한 이동

세션 토큰 발급과 해시 저장 (Spring Boot)

```
byte[] bytes = new byte[32];
RANDOM.nextBytes(bytes);
String token = HexFormat.of().formatHex(bytes);
Duration age = Boolean.FALSE.equals(input.keepLoggedIn())
    ? Duration.ofDays(1) : SESSION_AGE;
jdbc.update("DELETE FROM sessions WHERE expires_at < ?", Instant.now());
jdbc.update("INSERT INTO sessions (...) VALUES (?, ?, ?)",
    hash(token), userId, Instant.now().plus(age));
```

오픈 리다이렉트 차단 (프론트엔드)

```
const destinations = new Set(['/dashboard', '/exercise', '/places', '/favorites', '/mypage'])

export function safeReturnTo(value: string | null): string {
  if (!value || !value.startsWith('/') || value.startsWith('//') ||
      /[\\\s]/.test(value)) return '/dashboard'
  const path = value.split(/[?#]/, 1)[0]
  return destinations.has(path) ? value : '/dashboard'
}
```

세션 토큰 발급

로그인에 성공하면 세션을 만들어 DB에 저장합니다.

1. 32바이트 랜덤 토큰 생성
2. DB에는 SHA-256 해시만 저장
3. 로그인 유지 해제 시 1일, 기본 30일

오픈 리다이렉트 차단

로그인 후 돌아갈 주소를 안전한 내부 경로로 제한합니다.

1. //, 백슬래시, 공백이 있으면 거부
2. 허용 목록 5개 경로만 통과
3. 그 외에는 /dashboard로 이동

위치 조회: 오래된 응답 무시

위치 조회 경쟁 상태 방지 (프론트엔드)

```
const locate = useCallback(() => {
  const id = ++requestId.current
  ...
  navigator.geolocation.getCurrentPosition(position => {
    if (id !== requestId.current) return
    if (!isUsableLocation(position.coords, requireAccurateLocation ? 100 : Infinity)) {
      setError(inaccurateLocationMessage(position.coords.accuracy)); setLoading(false); return
    }
    setCoordinates({ latitude: position.coords.latitude, longitude: position.coords.longitude })
    setLoading(false)
  }, failure => {
    if (id !== requestId.current) return
    setError(failure.code === 1 ? '위치 권한이 거부되었습니다. …'
      : failure.code === 3 ? '위치 확인 시간이 초과되었습니다. …' : '위치를 확인할 수 없습니다. …')
    setLoading(false)
  }, { enableHighAccuracy: requireAccurateLocation, timeout: 20000, maximumAge: 0 })
}, [requireAccurateLocation])
```

위치 조회 경쟁 상태 방지

브라우저 위치를 조회하고, 오래된 응답은 무시합니다.

1. 요청 번호(requestId)로 최신 응답만 반영
2. 거부·시간초과·기타 오류별 안내 문구
3. 타임아웃 20초, 캐시된 위치는 쓰지 않음

위치를 여러 번 요청해도 마지막 요청의 결과만 화면에 반영되어, 늦게 도착한 이전 응답이 최신 위치를 덮어쓰지 않습니다.

상황별 안내

권한 거부
위치 권한이 거부되었습니다.

시간 초과 (20초)
위치 확인 시간이 초과되었습니다.

기타 오류
위치를 확인할 수 없습니다.

정확도 부족
정확도가 낮으면 별도 안내 후 중단

주변 장소 병렬 조회와 날씨 기반 필터

주변 장소 병렬 조회 (프론트엔드)

```
Promise.all([
  Promise.all(exercises.map(item =>
    getPlaces(location.coordinates!, item, controller.signal, 15))),
  getRegion(location.coordinates, controller.signal),
]).then(([placeGroups, region]) => {
  const unique = [...new Map(
    placeGroups.flat().map(place => [place.id, place])).values()]
    .sort((a, b) => (a.distance ?? Number.MAX_SAFE_INTEGER)
      - (b.distance ?? Number.MAX_SAFE_INTEGER))
  setPlaces(unique)
  setAddress(region.address)
})
// 비가 오면 기본 필터를 '실내'로 (사용자가 바꿨다면 유지)
getWeather(location.coordinates, controller.signal).then(weather => {
  if (!controller.signal.aborted && !filterTouched.current) {
    setFilter(/비|소나기|빗방울|강수/.test(weather.condition) ? '실내' : '전체')
  }
})
```

주변 장소 병렬 조회

다섯 가지 운동의 장소를 한꺼번에 불러와 합치고, 날씨에 맞는 초기 필터를 정합니다.

1. 운동 5종 × 최대 15곳을 병렬 조회
2. Map으로 id 중복 제거 후 거리순 정렬
3. 비가 오면 '실내' 필터가 기본

사용자가 필터를 직접 바꾼 뒤에는 날씨가 바뀌어도 선택한 필터를 유지합니다.



이동 경로 기록: GPS 노이즈 필터와 저장 큐

GPS 노이즈 필터

```
export function acceptedMovementMeters(
  from: MovementPoint, to: MovementPoint, accuracy: number) {
  if (!Number.isFinite(accuracy) || accuracy < 0) return 0
  const radians = (degrees: number) => degrees * Math.PI / 180
  const a = Math.sin(radians(to.latitude - from.latitude) / 2) ** 2
    + Math.cos(radians(from.latitude)) * Math.cos(radians(to.latitude))
    * Math.sin(radians(to.longitude - from.longitude) / 2) ** 2
  const segment = 6371000 * 2 * Math.asin(Math.sqrt(Math.min(1, Math.max(0, a))))
  return segment >= Math.max(5, Math.min(50, accuracy * .5))
    && segment < 1000 ? segment : 0
}
```

이동 경로 저장 큐

```
const flush = useCallback(async () => {
  ...
  saving.current = controller
  while (pending.current.length && !controller.signal.aborted) {
    const point = pending.current[0]
    try {
      const saved = await saveRoutePoint(point, controller.signal)
      pending.current.shift()
      setRoutes(previous => /* id로 중복 확인 후 날짜별 병합 */)
    } catch (reason) {
      setError(...)
      if (reason.message.includes('로그인')) { pending.current = []; setAuth('guest') }
      break
    }
  }
}, [auth, account])
// 5초마다: if (pending.current.length) void flush()
```

GPS 노이즈 필터

두 위치 사이 거리를 구해 실제 이동만 누적합니다.

1. Haversine 공식으로 거리 계산
2. 최소 이동 max(5m, min(50m, 오차×0.5))
3. 1km 이상 튀는 값은 제외

이동 경로 저장 큐

위치 점을 Spring Boot 서버에 순서대로 안전하게 저장합니다.

1. 성공한 점만 큐에서 제거
2. 실패하면 멈추고 5초 뒤 재시도
3. 로그인이 풀리면 게스트 상태로 전환

이동 경로 저장: 멱등 저장과 자동 재시도

경로 점 멱등 저장 (Spring Boot)

```
UUID userId = sessionAuth.requireUser(token);
validate(point);
LocalDate date = point.recordedAt().atZone(KOREA).toLocalDate();
try {
    jdbc.update("INSERT INTO route_points (...) VALUES (?, ?, ?, ?, ?, ?)",
        point.id(), userId, date, point.recordedAt(), point.latitude(), point.longitude());
} catch (DuplicateKeyException duplicate) {
    Integer existing = jdbc.queryForObject(
        "SELECT COUNT(*) FROM route_points WHERE id = ? AND user_id = ?",
        Integer.class, point.id(), userId);
    if (existing == null || existing == 0)
        throw new ResponseStatusException(HttpStatus.CONFLICT, "위치 기록 ID가 이미 사용 중입니다.");
}
```

경로 API 재시도 (프론트엔드)

```
for (let attempt = 0; attempt < 3; attempt++) {
  init?.signal?.throwIfAborted()
  try {
    const response = await fetch(`/api/auth/routes${path}`, { credentials: 'same-origin', ...init })
    ...
    if (response.status === 401) throw new RouteError('이동 경로를 저장하려면 로그인해 주세요.', false)
    if (!response.ok && response.status < 500) throw new RouteError(data?.error || '…', false)
    if (!response.ok || !data) throw new RouteError('서버 연결이 잠시 끊겼습니다. …', true)
    return data as T
  } catch (error) {
    init?.signal?.throwIfAborted()
    if (error instanceof RouteError && !error.retryable) throw error
    if (attempt === 2) throw new Error('서버 연결이 잠시 끊겼습니다. …')
    await new Promise(resolve => setTimeout(resolve, 400 * (attempt + 1)))
  }
}
```

경로 점 멱등 저장

위치 점을 저장하되, 같은 요청이 다시 와도 한 번만 저장되게 합니다.

1. 클라이언트가 만든 UUID가 기본 키
2. 중복이면 성공 처리, 남의 ID와 충돌하면 409
3. 저장 시각을 KST 날짜로 변환

경로 API 재시도

서버 연결이 잠시 끊겨도 자동으로 다시 시도합니다.

1. 최대 3회, 대기 시간 0.4초 → 0.8초로 증가
2. 4xx(로그인 필요 등)는 즉시 실패
3. 5xx·네트워크 오류만 재시도

즐겨찾기: 저장 검증과 카테고리 분류

즐겨찾기 저장, 입력 검증 (Spring Boot)

```
jdbc.update("""
    MERGE INTO favorite_places (user_id, place_id, ..., created_at)
    KEY (user_id, place_id)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, userId, placeId, place.name().trim(), ...);

if (place == null || placeId.isBlank() || placeId.length() > 100 ||
    !placeId.equals(place.id()) || invalidText(place.name(), 200) ||
    (place.latitude() != null && (!Double.isFinite(place.latitude()) ||
        place.latitude() < -90 || place.latitude() > 90)) ||
    (place.url() != null && (place.url().length() > 500 ||
        !place.url().matches("https?://place\\.map\\.kakao\\.com/[^\\s]+")))) {
  throw new ResponseStatusException(HttpStatus.BAD_REQUEST, "올바른 장소 정보를 보내주세요.");
}
```

장소 카테고리 분류 (프론트엔드)

```
function categoryOf(place: FavoritePlace) {
  const classify = (text: string) => {
    if (/주차|화장실|편의점|매점|휴게소|대여소/.test(text)) return '편의시설'
    if (/수영|체육|스포츠|운동장|…|태권도|자전거/.test(text)) return '스포츠·체육'
    if (/산책|등산|둘레길|탐방로|트레킹/.test(text)) return '산책·등산'
    if (/공원|숲|수목원|식물원|자연|유원지|산$|계곡|해수욕장/.test(text)) return '공원·자연'
    return null
  }
  // Prefer the specific facility category over words in the place name.
  for (const category of place.category.split('>').reverse()) {
    const group = classify(category.trim())
    if (group) return group
  }
  return classify(place.name) ?? '기타'
}
```

저장 전 입력 검증

저장 전에 입력을 검증하고, 이미 있으면 갱신·없으면 추가합니다.

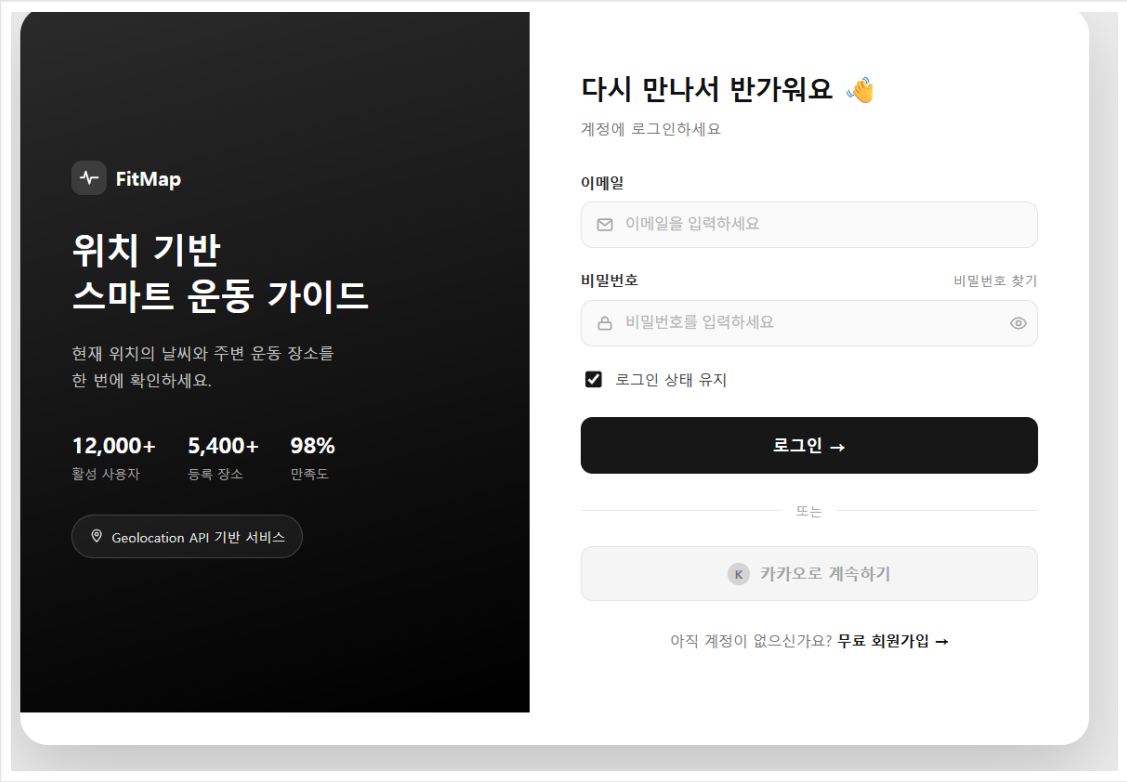
1. MERGE … KEY로 upsert해 중복 저장 방지
2. URL의 ID와 본문의 ID 일치 확인
3. 위경도 범위·길이·카카오 URL만 허용

장소 카테고리 분류

카카오 카테고리 문자열을 앱의 5개 그룹으로 분류합니다.

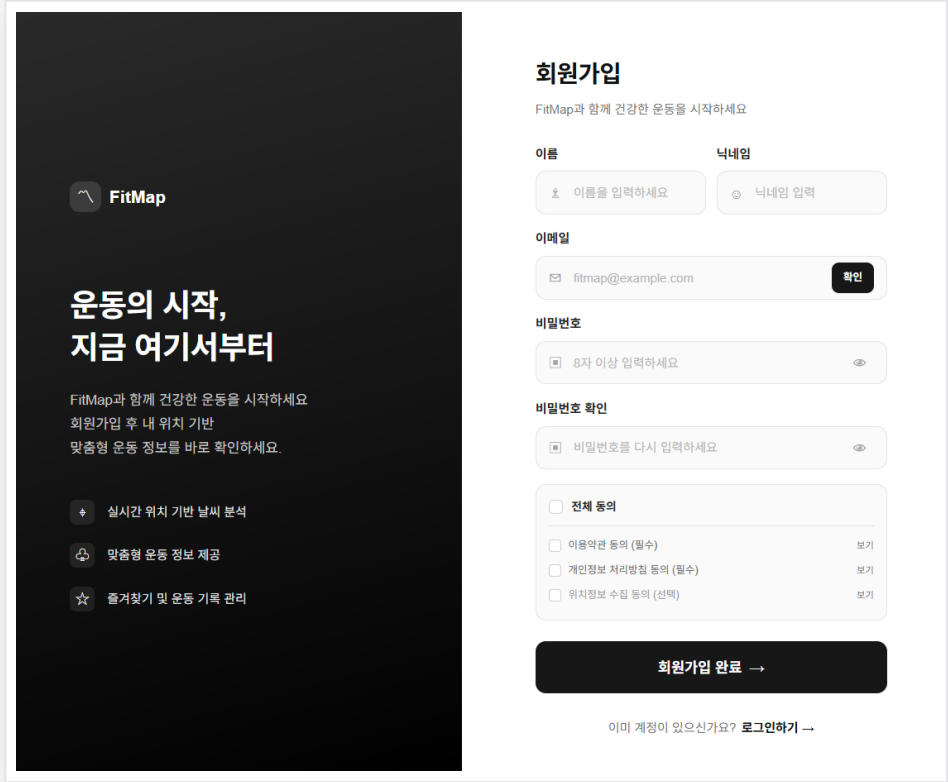
1. '>' 뒤쪽부터 보며 구체적인 시설 분류 우선
2. 못 정하면 장소 이름으로 재분류
3. 그래도 없으면 '기타'

UI 구성 (시안): 로그인과 회원가입



로그인 화면

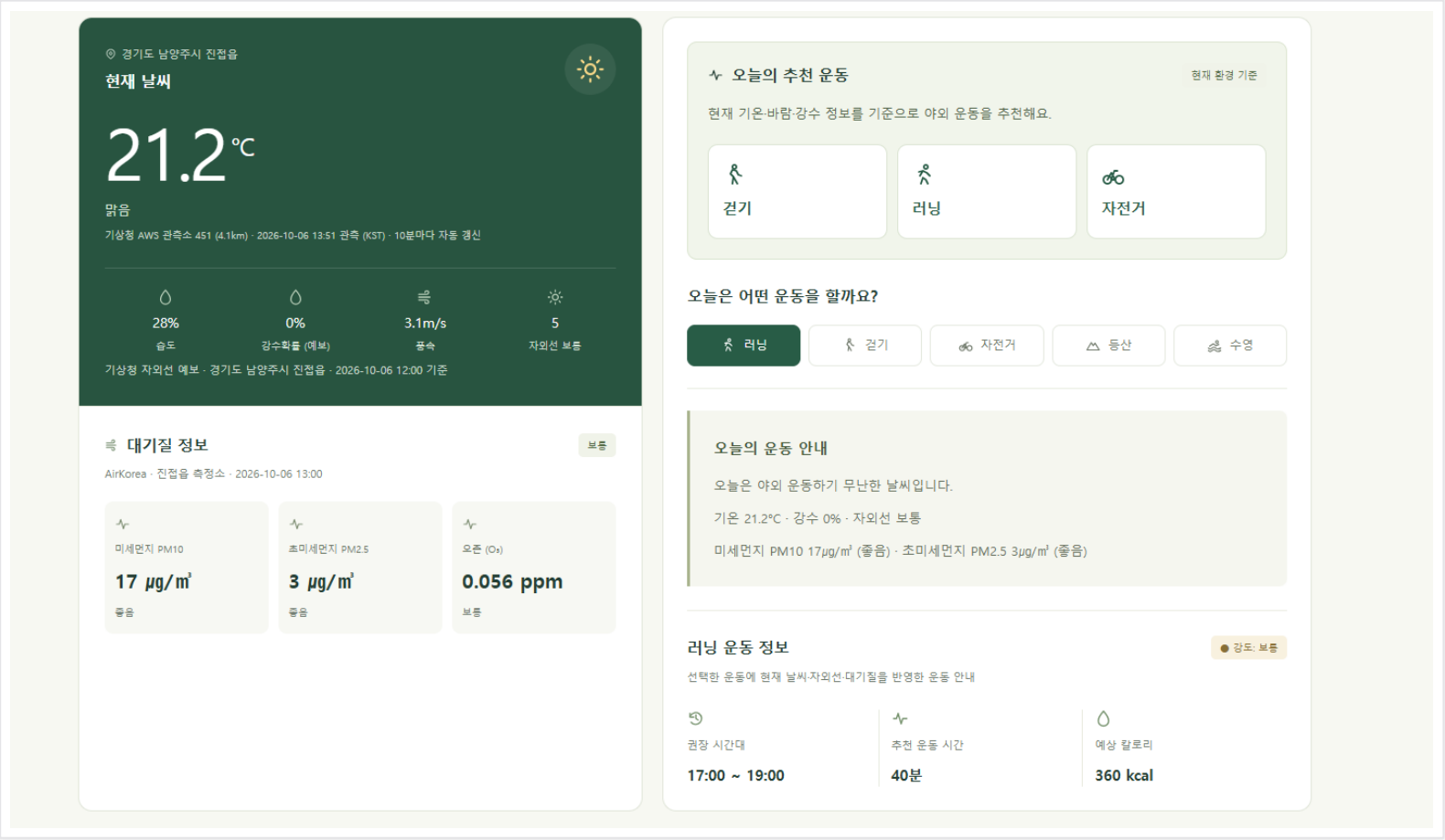
이메일과 비밀번호 입력, 로그인 상태 유지, 카카오 계정 로그인



회원가입 화면

이름, 닉네임, 이메일, 비밀번호 확인, 약관 동의

UI 구성: 대시보드



현재 날씨

기상청 AWS 관측소 실측값, 10분마다 자동 갱신

습도, 강수확률, 풍속, 자외선

기상청 예보와 자외선 예보 기준

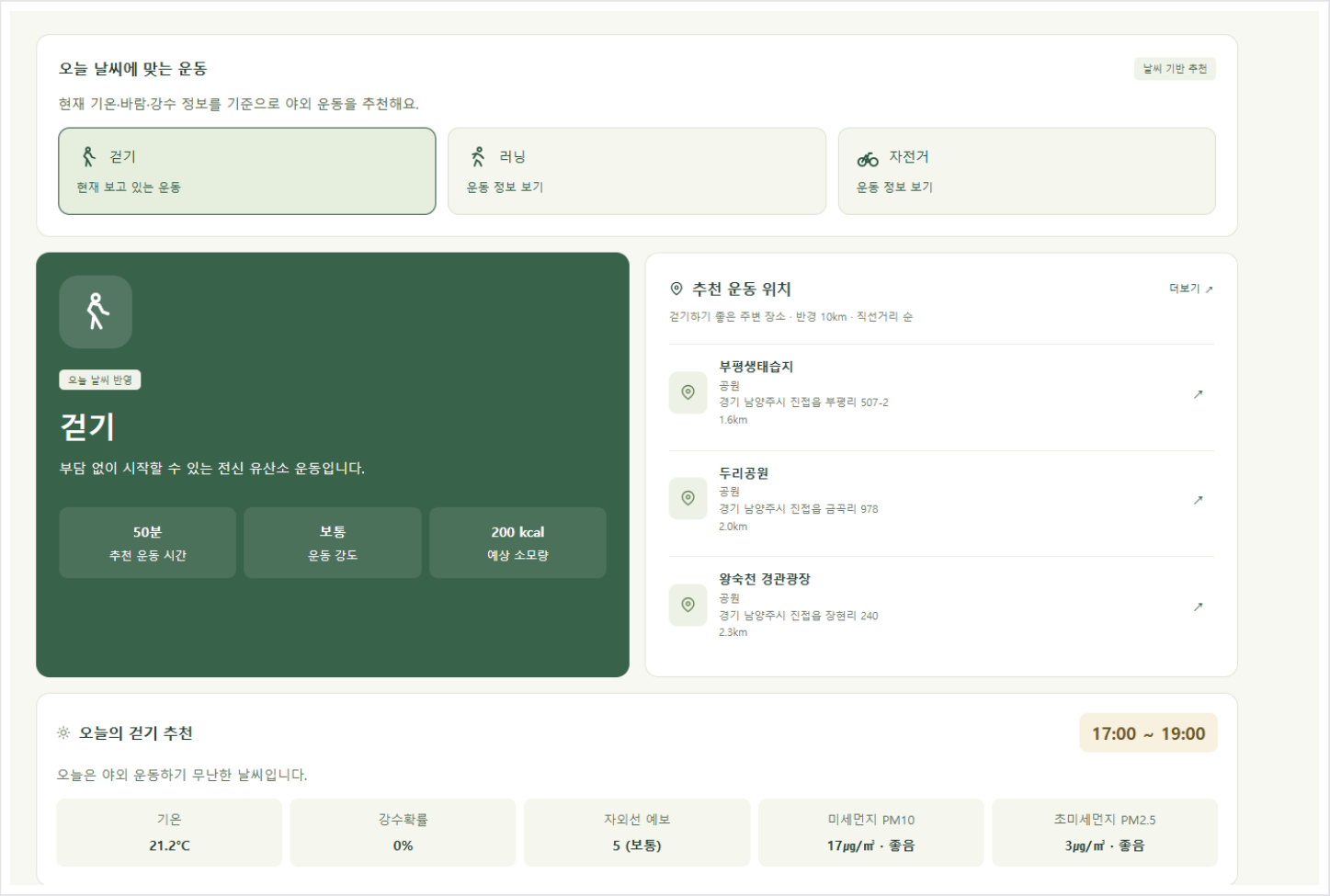
대기질 정보

에어코리아 측정소의 PM10, PM2.5, 오존

오늘의 추천 운동

종목을 고르면 권장 시간대, 추천 운동 시간, 예상 칼로리 안내

UI 구성: 운동정보 페이지



오늘 날씨에 맞는 운동

기온, 바람, 강수 기준으로 걷기, 러닝, 자전거 중 추천

선택한 운동 요약

추천 운동 시간, 운동 강도, 예상 소모 칼로리

추천 운동 위치

반경 10km 안의 주변 장소를 직선거리 순으로 표시

오늘의 추천

권장 시간대와 기온, 강수확률, 자외선, 미세먼지 요약

시스템 활용 방안



맑은 날, 야외 운동

야외에서 운동할 때 사용자 위치를 기반으로 주변의 운동하기 좋은 장소와 날씨를 한눈에 표시합니다.

1 현재 위치 확인

2 날씨와 미세먼지 확인

3 주변 야외 운동 장소 표시



악천후, 실내 운동

악천후에도 실내에서 운동할 수 있도록 실내 운동 장소를 지도에 표시해, 날씨와 관계없이 운동할 수 있습니다.

1 우천, 미세먼지 나쁨 감지

2 실내 장소로 전환

3 지도에서 바로 확인

기대 효과 및 결론

결론

날씨, 미세먼지, 주변 운동 장소를 한곳에 모아 날씨에 흔들리지 않는 운동을 가능하게 합니다.

공공데이터 API | Kakao API | React | Node.js | Spring Boot | Docker

정보 탐색 효율성 극대화

날씨, 미세먼지, 주변 운동장 정보를 단일 플랫폼에서 제공해 정보를 모으는 데 드는 시간을 크게 줄입니다.

운동 연속성과 안전성 확보

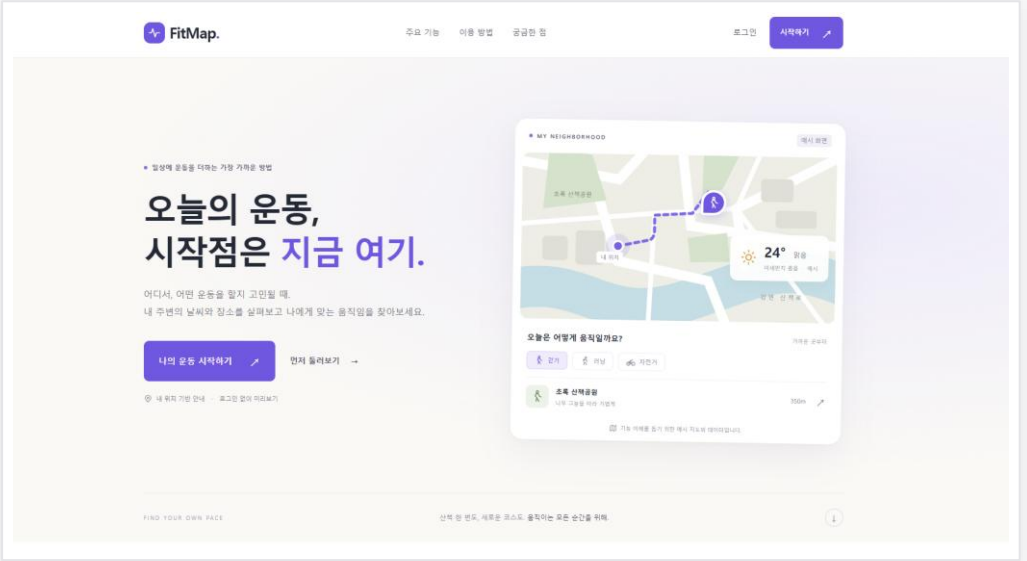
우천, 폭염, 고농도 미세먼지 같은 실시간 기상 상태에 맞는 장소를 추천받아 날씨 제약 없이 안전하게 운동합니다.

위치 기반 편의성 증대

카카오 API로 현재 위치 주변의 가까운 운동 장소를 빠르게 파악할 수 있습니다.

감사합니다

질문을 받겠습니다



FitMap 주승재, 신동환, 이정윤, 최우혁